PACKAGE MANAGER FOR DEBIAN BASED DISTRIBUTIONS

A report submitted in partial fulfilment of the requirements

for the award of the degree of

B.Tech Computer Science and Engineering

by

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DESIGN AND MANUFACTURING KURNOOL

November 2025

Evaluation Sheet

Title of the Project: Package Manager For Debian Based Distributions

Name of the Student(s):

P Sanjay Davis - 123CS0031

Y Sri Rama Bharadwaj - 123CS0030

Examiner(s):

Supervisor(s):

Head of the Department:

Date:

Place:

Certificate

We, **P Sanjay Davis (123CS0031)** and **Y Sri Rama Bharadwaj (123CS0030)** hereby declare that the material presented in the Project Report titled **Package Manager For Debian Based Distributions** represents original work carried out by us in the **Department of Computer Science and Engineering** at the **Indian Institute of Information Technology Design and Manufacturing Kurnool** during the year **2025**. With our signatures, we certify that:

- We have not manipulated any of the data or results.
- We have not committed any plagiarism of intellectual property. We have clearly indicated and referenced the contributions of others.
- We have explicitly acknowledged all collaborative research and discussions.
- We have understood that any false claim will result in severe disciplinary action.
- We have understood that the work may be screened for any form of academic misconduct.

Date: 06- 11 -25 Student Signatures: P Sanjay Davis, Y Sri Rama Bharadwaj

In my capacity as supervisor of the above-mentioned work, I certify that the work presented in this Report is carried out under my supervision, and is worthy of consideration for the requirements of B.Tech. Project work.

Advisor's Name: Srinivas Naik Advisor's Signature

Abstract

This report details the development of a Package Manager For Debian Based Distributions, a robust system designed to streamline the management of Ubuntu packages. The core of this system is a MySQL database engineered for high-performance retrieval and tracking of over 100,000 Ubuntu packages and their dependencies(∼235,000 relationships).

The project is structured into three main components:

1. Frontend (React Admin Dashboard):A modern, dark-themed web interface for administrators to perform real-time package search, view dependency graphs, and monitor user installation logs stored in the database.
2. Backend (Node.js/Express RESTful API):Manages all database interactions (search, retrieval, logging) and provides secure JWT-authenticated endpoints for both the admin dashboard and the CLI tool. It is also responsible for real-time syncing of package data from Ubuntu repositories.
3. CLI Tool (Python):A user-facing command-line interface that features smart dependency resolution, version compatibility checks, and logging of all installation metrics back to the central database. This component ensures system-level integrity with features like `t64` conflict protection.

The Database Management System (DBMS), specifically MySQL, is the foundational element, utilizing an optimized schema with four primary tables (`users`, `packages`, `dependencies`, `packagedownloads`) and strategic indexing for sub-100ms API response times. The report elaborates on the database design, the multi-component architecture, and the implementation of a full CRUD cycle for package data and download logs, demonstrating a comprehensive and efficient solution for package management.

Keywords: DBMS, MySQL, Database Schema, Package Manager, Ubuntu, Dependency Resolution, Full-Stack Development, React, Node.js, Python CLI, JWT Authentication.

Acknowledgements

We express our sincere gratitude to our project guide, Dr. Srinivas Naik, Assistant Professor, Department of Computer Science and Engineering, IIITDM Kurnool, for his insightful guidance, and unwavering support. His expertise was crucial in the successful completion of this complex, multi-component project.

We are also thankful to the Head of the Department, Dr.Srinivas Naik, for providing the necessary resources.

Finally, we thank our teammates for their collaborative effort and dedication, and our family and friends for their constant encouragement.

Contents

Evaluation Sheet i

Certificate ii

Acknowledgements iv

Contents v

1 Introduction 1
- 1.1 Overview of the Project . . . 1
- 1.2 Motivation and Objectives . . . 1
- 1.3 Technology Stack . . . 2
- 1.4 Report Structure . . . 2

2 Database Design and Schema 3
- 2.1 Introduction to the DBMS . . . 3
- 2.2 Entity-Relationship (E-R) Model . . . 3
- 2.3 Optimized Schema Implementation (DDL) . . . 4
  - 2.3.1 Table: `users` . . . 4
  - 2.3.2 Table: `packages` . . . 4
  - 2.3.3 Table: `dependencies` . . . 5
  - 2.3.4 Table: `packagedownloads` . . . 5
- 2.4 Database Maintenance and Optimization . . . 6

3 Backend: RESTful API and Business Logic 7
- 3.1 Architecture and Technologies . . . 7
- 3.2 Security and Authentication . . . 7
  - 3.2.1 Role-Based Access Control (RBAC) . . . 8
  - 3.2.2 API Protection . . . 8
- 3.3 Core Data Endpoints . . . 8
  - 3.3.1 Package Retrieval and Search . . . 8
  - 3.3.2 Synchronization and Maintenance . . . 9
  - 3.3.3 Installation Logging . . . 9

3.4 Implementation Details . . . 9

4 Frontend: React Admin Dashboard **10**
4.1 Introduction and Architecture . . . 10
4.2 User Interface Design and Experience . . . 10
4.2.1 Modern Aesthetics and Responsiveness . . . 10
4.2.2 Admin Authentication Flow . . . 11
4.3 Key Administrative Views and Functionality . . . 11
4.3.1 Packages View: Catalog Management . . . 11
4.3.2 Dependencies View: Relationship Inspection . . . 12
4.3.3 Downloads View: Activity Monitoring . . . 12
4.4 Frontend Code Structure . . . 12

5 CLI Tool: Python-Based Package Installation Utility **13**
5.1 Design and Architecture . . . 13
5.2 User Authentication and Configuration . . . 13
5.2.1 Secure Credential Management . . . 13
5.2.2 Configuration and Environment . . . 14
5.3 Core Package Management Commands . . . 14
5.3.1 Information Retrieval . . . 14
5.3.2 Intelligent Installation Logic (`install`) . . . 14

6 Implementation Details and Performance **16**
6.1 System Architecture Overview . . . 16
6.1.1 Project Structure and Modularity . . . 16
6.1.2 Inter-Component Communication . . . 17
6.2 Database Performance and Optimization . . . 17
6.2.1 Indexing Strategy . . . 17
6.2.2 API Response Time Statistics . . . 18
6.3 Reliability and Troubleshooting . . . 18
6.3.1 Handling Version Conflicts . . . 18
6.3.2 Network Configuration Fixes . . . 18

7 Conclusion **19**
7.1 Conclusion . . . 19
7.2 Summary of Key Achievements . . . 20

A Appendix A: Installation and Setup Guide **21**
A.1 Prerequisites . . . 21
A.2 Database Setup . . . 21
A.2.1 Schema Creation . . . 21
A.2.2 Data Import . . . 22
A.3 Backend (API) Installation . . . 22
A.4 Frontend (Admin Dashboard) Installation . . . 23

A.5 CLI Tool Setup 23
A.5.1 Testing Basic Functionality 23

B Appendix B: RESTful API Documentation Summary 24
B.1 Introduction 24
B.2 Authentication and Security Endpoints 24
B.3 Package Data Endpoints (Read Operations) 25
B.4 Logging and Administration Endpoints 25
B.5 Data Schemas 25

C Appendix C: Troubleshooting and Common Issues 26
C.1 Database and Backend Connectivity Issues 26
C.1.1 Issue 1: Connection Refused (WSL/Windows Scenario) 26
C.1.2 Issue 2: MySQL Access Denied 27
C.2 Package Data Integrity and Synchronization 27
C.2.1 Issue 3: Incomplete Dependency Graph 27
C.2.2 Issue 4: Package Transition Conflicts (`t64`) 27
C.3 User Interface (Frontend) Errors 28
C.3.1 Issue 5: Downloads View Data Latency 28

Chapter 1

Introduction

1.1 Overview of the Project

This project develops a "Package Manager for Debian Based Distributions" which includes an Admin Dashboard. It is designed as a comprehensive, full-stack system for managing Debian-based Linux packages, featuring an intelligent Command Line Interface (CLI) and a web-based administrative panel. The entire system is built upon a robust MySQL Database Management System (DBMS), which acts as the central data repository for all package metadata, dependency relationships, and critical user installation logs.

1.2 Motivation and Objectives

The primary motivation for this project is to create an efficient, transparent, and feature-rich package management utility that offers centralized administrative oversight and detailed logging capabilities. The key objectives are:

- To design and implement a highly optimized relational database schema capable of managing the complexity and sheer volume of Debian package records ($\approx 100,000$ packages) and their dependencies ($\approx 235,000$ relationships).

- To develop a smart CLI tool (Python) that performs advanced tasks such as recursive dependency resolution, version compatibility checking, and protection against known system conflicts (`t64` transition issues).
- To build a modern web-based Admin Dashboard (React) for visualization, real-time package search, and monitoring of all user activity and database synchronization metrics.
- To implement a secure RESTful API (Node.js/Express) with JWT authentication to mediate all transactions between the client layers and the high-performance MySQL database.

1.3 Technology Stack

The system employs a multi-language, multi-tier architecture, integrating modern technologies for a robust solution:

- **Database:** MySQL 8.0+ (DBMS)
- **Backend/API:** Node.js with Express
- **Frontend/Admin Dashboard:** React
- **CLI Tool/Utility Scripts:** Python 3.8+

1.4 Report Structure

The remainder of this report details the system's construction: Chapter 2 focuses on the Database Design and Schema; Chapter 3 covers the Backend API; Chapter 4 describes the React Frontend; Chapter 5 presents the Python CLI Tool; Chapter 6 discusses implementation, performance, and testing; and Chapter 7 provides the conclusion

Chapter 2

Database Design and Schema

2.1 Introduction to the DBMS

The MySQL 8.0 database is the foundational component of the Package Manager for Debian Based Distributions, serving as the single source of truth for all package metadata, dependency relationships, and user logs. The core design principles for the database were data integrity, query performance, and scalability to efficiently manage the large volume of Ubuntu package information ($\approx 100,000$ packages).

2.2 Entity-Relationship (E-R) Model

The system's data model revolves around four central entities that form the basis of the relational schema:

1. **Users:** Stores secure authentication data for administrators and general CLI users.
2. **Packages:** The master catalog of all Debian/Ubuntu packages and their associated details.
3. **Dependencies:** A relational entity defining the many-to-many relationship between packages (i.e., which package `Depends` on which other packages).

4. **Package Downloads:** A logging mechanism that records installation metrics, statuses, and timestamps for audit and monitoring purposes.

2.3 Optimized Schema Implementation (DDL)

The database, named `adminpy`, utilizes the following schema. Strategic indexing is applied to columns that are frequently used for search and join operations, which is vital for the low latency required by the Node.js API.

2.3.1 Table: `users`

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(50) DEFAULT 'user'
);
```

Listing 2.1: Schema for the Users Table

2.3.2 Table: `packages`

This table stores approximately 100,000 package records. The index on the `name` column is critical for supporting the real-time search functionality required by both the CLI and the Admin Dashboard.

```
CREATE TABLE packages (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    version VARCHAR(255),
    architecture VARCHAR(50),
    filename VARCHAR(1000),
    INDEX idx_name (name)
);
```

Listing 2.2: Schema for the Packages Table

2.3.3 Table: `dependencies`

This is the core relationship table, tracking approximately 235,000 package requirements. It links packages to their dependencies via a foreign key and includes an index on the dependency name for efficient recursive lookups during the CLI's dependency resolution process.

```
CREATE TABLE dependencies (
    id INT AUTO_INCREMENT PRIMARY KEY,
    package_id INT NOT NULL,
    dependency_name VARCHAR(255) NOT NULL,
    version_constraint VARCHAR(1000),
    FOREIGN KEY (package_id) REFERENCES packages(id) ON DELETE CASCADE,
    INDEX idx_package_id (package_id),
    INDEX idx_dependency_name (dependency_name)
);
```

Listing 2.3: Schema for the Dependencies Table

2.3.4 Table: `packagedownloads`

This logging table records every installation event, providing a transparent audit trail for administrators.

```
CREATE TABLE packagedownloads (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id VARCHAR(255) NOT NULL,
    package_name VARCHAR(255) NOT NULL,
    version VARCHAR(255),
    download_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    download_duration_seconds DECIMAL(10, 2),
    install_duration_seconds DECIMAL(10, 2),
    client_ip VARCHAR(45),
    download_status VARCHAR(50),
    install_status VARCHAR(50),
    INDEX idx_user_id (user_id),
    INDEX idx_package_name (package_name),
    INDEX idx_timestamp (download_timestamp)
);
```

Listing 2.4: Schema for the Package Downloads Table

2.4 Database Maintenance and Optimization

Performance is maintained through the use of B-tree indexes on search columns. Furthermore, data integrity and disaster recovery are managed by utility Python scripts (`save_db.py` and `restore_db.py`) that automate compressed Backup and Restore operations for the entire `admin.py` database.

Chapter 3

Backend: RESTful API and Business Logic

3.1 Architecture and Technologies

The backend is the critical middleware layer of the Package Manager for Debian Based Distributions, implemented using Node.js with the Express framework. It serves two main functions: providing data to the Admin Dashboard (React) and supplying package intelligence to the CLI tool (Python). It is responsible for all secure communication with the MySQL database, utilizing connection pooling to manage high-volume, concurrent requests efficiently.

3.2 Security and Authentication

Security is managed through JSON Web Tokens (JWT) for session integrity and authorization across the entire system.

3.2.1 Role-Based Access Control (RBAC)

The system distinguishes between administrative and user access via separate login pathways, which dictates access privileges for subsequent API calls.

- `POST /login`: Endpoint dedicated exclusively for administrator authentication (used by the React Frontend).
- `POST /login-cli`: Endpoint used by all general users accessing the Python CLI tool, granting read-only access to package data and write access only for download logs.

3.2.2 API Protection

All data endpoints require a valid JWT via the `Authorization: Bearer <token>` header. Additionally, Rate Limiting is applied to prevent abuse and ensure system stability, particularly on resource-intensive endpoints like search.

3.3 Core Data Endpoints

The API facilitates the four main types of interactions with the DBMS, supporting the core functionality of the package manager.

3.3.1 Package Retrieval and Search

- `GET /api/packages`: Retrieves a list of packages from the `packages` table. It supports query parameters for real-time searching and pagination, demonstrating the performance of the indexed MySQL schema.
- `GET /api/dependencies`: Fetches the dependency relationship records for a specified package, directly querying the `dependencies` table to supply data for the CLI's resolution logic.

3.3.2 Synchronization and Maintenance

- `POST /api/update-packages` (Admin Only): This is an administrative endpoint that executes a critical bulk data operation. It triggers the process of downloading the latest package lists from configured Ubuntu repositories and parsing this data for insertion/update into the `packages` and `dependencies` tables in MySQL. This process ensures the database remains synchronized with upstream package sources.

3.3.3 Installation Logging

- `POST /api/log-download`: Called by the Python CLI tool to record a package installation event. It writes detailed telemetry data—including user ID, package name, download duration, and installation status—to the `packagedownloads` table.
- `GET /api/packagedownloads`: Used by the Admin Dashboard to fetch the historical installation logs for monitoring and audit purposes.

3.4 Implementation Details

The Express application is structured to handle asynchronous database operations efficiently, ensuring low latency. Proper error handling is implemented across all endpoints, providing meaningful feedback to the client applications while protecting the internal database structure.

Chapter 4

Frontend: React Admin Dashboard

4.1 Introduction and Architecture

The frontend serves as the centralized administrative panel for the Package Manager for Debian Based Distributions. It is built as a Single Page Application (SPA) using the React framework, providing a modern, data-driven interface for system monitoring and management. It interacts with the backend solely via the protected RESTful API endpoints.

4.2 User Interface Design and Experience

4.2.1 Modern Aesthetics and Responsiveness

The dashboard utilizes a distinct dark theme with gradient accents to ensure visual appeal and reduced eye strain during prolonged use. The design employs responsive techniques to ensure full usability and proper rendering across various screen sizes, including desktop, tablet, and mobile browsers.

4.2.2 Admin Authentication Flow

Access to the dashboard is strictly controlled.

1. The administrator initiates login using the credentials provided to the backend's `/login` endpoint.
2. Upon successful authentication, a JSON Web Token (JWT) is received and stored securely.
3. This JWT is attached to all subsequent API calls in the `Authorization: Bearer` header, maintaining a persistent, secure session for administrative tasks.

4.3 Key Administrative Views and Functionality

The dashboard provides three primary views, all driven by data retrieved in real-time from the MySQL database through the Node.js API:

4.3.1 Packages View: Catalog Management

This view provides comprehensive tools for package catalog exploration:

- **Real-time Search:** Enables administrators to quickly search and filter the ∼100,000 package records, leveraging the high performance of the indexed MySQL `packages` table.
- **Pagination:** Manages the display of the large package catalog efficiently, retrieving and presenting results in digestible chunks via API query parameters.
- **One-Click Package Update:** A key admin function that sends a request to the highly privileged `POST /api/update-packages` endpoint, initiating the synchronization process with upstream Ubuntu repositories to refresh the central database.

4.3.2 Dependencies View: Relationship Inspection

This view is dedicated to visualizing and inspecting the complex dependency graph. It allows administrators to query specific packages and view their direct and nested dependencies as recorded in the `dependencies` table, which is crucial for troubleshooting installation issues.

4.3.3 Downloads View: Activity Monitoring

This acts as the system's audit log viewer, retrieving and displaying data from the `packagedownloads` table.

- **Installation Audit:** Provides a chronological list of all package installations or attempts made by CLI users.
- **Performance Metrics:** Displays key performance indicators logged by the CLI, such as `download_timestamp`, `download_duration_seconds`, and the final `install_status`.

4.4 Frontend Code Structure

The React application is structured to ensure clarity and maintainability: components like `Login.js`, `Dashboard.js`, `PackagesTab.js`, and `DownloadsTab.js` are logically separated, aligning with the functional divisions of the administrative tasks.

Chapter 5

CLI Tool: Python-Based Package Installation Utility

5.1 Design and Architecture

The Command Line Interface (CLI) tool is the primary user-facing component of the Package Manager for Debian Based Distributions. It is developed entirely in Python 3.8+ and acts as a sophisticated client to the Node.js Backend API. Its design focus is on providing robust, intelligent package management capabilities while ensuring all operational data is reliably logged back into the central MySQL database.

5.2 User Authentication and Configuration

5.2.1 Secure Credential Management

The CLI handles user identity and session management through several commands:

- `register`: Allows a new user to create an account by calling the backend's `/register` API endpoint.

- `login`: Authenticates a user against the `/login-cli` endpoint. Upon success, the user's token and credentials are auto-saved to a local `.env` file (`PKG_MANAGER_USER`, `PKG_MANAGER_PASS`) to enable seamless subsequent interaction.
- `logout`: Clears the stored credentials from the `.env` file, ending the CLI session.

5.2.2 Configuration and Environment

The CLI relies on the local system environment, particularly when dealing with the underlying operating system (Ubuntu/WSL). Troubleshooting logic is incorporated, such as providing explicit instructions for resolving Connection Refused errors (e.g., updating the backend URL to the Windows IP address) when running the tool across different environments.

5.3 Core Package Management Commands

5.3.1 Information Retrieval

All information commands query the MySQL database via the API for real-time data:

- `search <package_name>`: Fetches and displays packages matching the search term, utilizing the high-speed search capabilities of the indexed `packages` table.
- `info <package_name>`: Retrieves the full package details, including its current version, architecture, and a list of all required dependencies as recorded in the `dependencies` table.

5.3.2 Intelligent Installation Logic (`install`)

The `install` command is the core operational feature, demonstrating advanced algorithms for system integrity:

1. **Smart Dependency Resolution:** It recursively analyzes the package dependency tree by querying the backend API, determining the minimum set of packages required for installation.

2. **Version Compatibility Check:** Before execution, it verifies version constraints to prevent installing incompatible packages, a crucial step for maintaining system stability.

3. **Conflict Protection:** Specific mechanisms are included to detect and block known system-breaking issues, such as conflicts related to the `t64` package transition in newer Ubuntu versions.

4. **Visual Feedback:** Provides Progress Bars to offer users real-time visual feedback during the package download process.

5. **Database Logging:** Every installation or attempted installation is tracked. The CLI makes calls to the `POST /api/log-download` endpoint, populating the `packagedownloads` table with detailed metrics, including installation duration and final status, serving as a comprehensive audit log.

Chapter 6

Implementation Details and Performance

6.1 System Architecture Overview

The Package Manager for Debian Based Distributions operates on a three-tier architecture, where the MySQL DBMS forms the data layer, the Node.js/Express API forms the application layer, and the React Dashboard and Python CLI form the presentation layer.

6.1.1 Project Structure and Modularity

The project is organized into clear, decoupled directories:

- `backend/`: Contains the Node.js API, handling routing, JWT middleware, and database connections.
- `frontend/`: Contains the React SPA, responsible for the administrator UI and data visualization.

- `debian_packages/`: A temporary directory used for storing downloaded `.deb` files during the CLI installation process.
- Root Level: Contains utility scripts (`restore_db.py`, `save_db.py`) and the main `test.py` CLI executable.

This modular structure simplifies development, deployment, and troubleshooting.

6.1.2 Inter-Component Communication

Communication is entirely managed via RESTful HTTP requests. This ensures that the components can be deployed independently, potentially across different machines (e.g., Python CLI on WSL/Ubuntu connecting to Node.js backend on a Windows host), provided proper network configuration (e.g., firewall rules, IP address configuration) is maintained.

6.2 Database Performance and Optimization

The efficiency of the entire system hinges on the performance of the MySQL data layer, especially given the scale ($\approx 100,000$ packages).

6.2.1 Indexing Strategy

The use of B-tree indexes on frequently queried columns dramatically reduces read latency:

- `packages.name`: Essential for fast search operations initiated by the CLI and Admin Dashboard.
- `dependencies.dependency_name` and `dependencies.package_id`: Crucial for rapid multi-table joins during dependency resolution.
- `packagedownloads.download_timestamp`: Enables quick filtering and sorting of installation logs for the administrative monitoring view.

6.2.2 API Response Time Statistics

Due to the optimized schema, the average API response time for package search and retrieval is consistently below 100ms. This high performance ensures a fluid user experience on both the CLI and the web dashboard, even when dealing with the full ∼500MB Noble repository dataset.

6.3 Reliability and Troubleshooting

Robustness is built into the system with specific solutions for common issues encountered during development.

6.3.1 Handling Version Conflicts

The CLI specifically addresses conflicts arising from package transitions (e.g., t64 transition conflicts in newer Ubuntu releases) by checking the database for compatible versions and warning the user, preventing system damage. This proactive conflict protection is a key advantage over simple manual installation.

6.3.2 Network Configuration Fixes

The most common deployment issue, Connection Refused (especially when the CLI client runs inside WSL and the backend runs on the Windows host), is addressed by explicit guidance to update the backend URL in the Python client to the appropriate Windows machine IP address, as detailed in the project documentation.

Chapter 7

Conclusion

7.1 Conclusion

The Package Manager for Debian Based Distributions successfully demonstrates a robust, full-stack solution for package management. The project effectively integrates a high-performance MySQL Database Management System (DBMS) with a modern application architecture consisting of a Node.js/Express API, a React Admin Dashboard, and a Python CLI tool.

The core success of this project lies in the optimized DBMS schema, which efficiently manages over 100,000 package records and their complex dependencies. This optimization allows the system to achieve consistently low API response times (sub-100ms) for critical operations like search and dependency lookups. The integration of advanced features in the CLI, such as smart dependency resolution and conflict protection, validates the practical utility of the system. In conclusion, the project meets all initial objectives, providing a secure, scalable, and high-performance alternative tool for managing Debian-based software packages with comprehensive administrative logging.

7.2 Summary of Key Achievements

- DBMS Foundation: Designed and deployed a four-table MySQL schema (`packages`, `dependencies`, `packagedownloads`, `users`) with strategic indexing for high-speed retrieval.
- Intelligent CLI: Developed a Python tool capable of recursive dependency resolution, version constraint checking, and automatic logging of all installations to the database.
- Centralized Monitoring: Created a React Admin Dashboard that provides real-time monitoring of user activity and simplifies high-volume data synchronization with Ubuntu repositories.
- Security and Integrity: Implemented JWT authentication across the stack and included specific logic (e.g., `t64` conflict protection) to ensure system stability during package installation.

Appendix A

Appendix A: Installation and Setup Guide

A.1 Prerequisites

To successfully run the Package Manager for Debian Based Distributions, the following software must be installed on the deployment machine (e.g., Ubuntu, WSL2):

- Database: MySQL 8.0+ Server and Client.
- Backend: Node.js (version 16 or higher) and npm.
- CLI: Python 3.8+ and pip.
- Tools: Git, and a method to run the frontend (e.g., a web browser).

A.2 Database Setup

A.2.1 Schema Creation

Log into your MySQL client and execute the commands to create the database and user:

```
CREATE DATABASE adminpy;
CREATE USER 'pkg\_manager'@'localhost' IDENTIFIED BY 'your\_secure\_password';
GRANT ALL PRIVILEGES ON adminpy.* TO 'pkg\_manager'@'localhost';
FLUSH PRIVILEGES;
```

Listing A.1: Database Setup

Note: Replace `'your_secure_password'` with your actual password.

A.2.2 Data Import

To populate the database with package data (if starting from a backup), use the utility script:

```
python restore_db.py --file [path_to_backup.sql.gz]
```

A.3 Backend (API) Installation

1. Navigate to the `backend/` directory.
2. Install dependencies:

   ```
   npm install
   ```

3. Configure the database connection string in the `.env` file (e.g., `DB_HOST`, `DB_USER`, `DB_PASS`).
4. Start the API server:

   ```
   npm start
   ```

   The API should now be running on the configured port (default 3000).

A.4 Frontend (Admin Dashboard) Installation

1. Navigate to the `frontend/` directory.
2. Install dependencies:

   ```
   npm install
   ```

3. Start the React development server:

   ```
   npm start
   ```

   The dashboard will open in your browser (default `http://localhost:3001`).

A.5 CLI Tool Setup

1. Ensure all Python requirements are met:

   ```
   pip install -r requirements.txt
   ```

2. The CLI tool is executed via `python test.py`. Use the `login` command first to authenticate and save credentials for session continuity.

A.5.1 Testing Basic Functionality

Once all components are running, test the system using the following commands:

```
python test.py login
python test.py search nano
python test.py install git
```

Verify the installation log appears on the Admin Dashboard's Downloads View.

Appendix B

Appendix B: RESTful API Documentation Summary

B.1 Introduction

The Node.js/Express backend provides a comprehensive set of RESTful API endpoints for managing the data stored in the MySQL database. All endpoints are hosted under the base URL `/api/`. Endpoints marked with **(Auth)** require a valid JWT in the `Authorization: Bearer <token>` header.

B.2 Authentication and Security Endpoints

Table B.1: Authentication Endpoints

Method	Endpoint	Access	Description
POST	`/login`	Admin Only	Authenticates an administrator and returns a JWT.
POST	`/login-cli`	CLI User	Authenticates a general CLI user and returns a JWT.
POST	`/register`	Public	Creates a new user entry in the `users` table.

B.3 Package Data Endpoints (Read Operations)

These endpoints are crucial for the Python CLI's intelligence and the React Dashboard's search functionality.

Table B.2: Package Retrieval Endpoints

Method	Endpoint	Auth	Description
GET	`/packages`	Yes	Retrieves a list of packages. Supports optional and .
GET	`/packages/:id`	Yes	Retrieves metadata for a specific package ID.
GET	`/dependencies/:id`	Yes	Retrieves the full list of dependencies for the specified .

B.4 Logging and Administration Endpoints

Table B.3: Logging and Administration Endpoints

Method	Endpoint	Access	Description
POST	`/log-download`	CLI User	Records a package installation attempt`packagedown` t
GET	`/packagedownloads`	Admin Only	Retrieves all installation history logs for
POST	`/update-packages`	Admin Only	Triggers the bulk data synchronization process

B.5 Data Schemas

All JSON responses follow standardized schemas. For instance, the `/log-download` endpoint expects the following body:

```
{
  "user_id": "123CS0031",
  "package_name": "git",
  "version": "2.39.2-1",
  "download_duration_seconds": 12.5,
  "install_duration_seconds": 25.8,
  "download_status": "SUCCESS",
  "install_status": "SUCCESS"
}
```

Listing B.1: Example POST /log-download Body

Appendix C

Appendix C: Troubleshooting and Common Issues

C.1 Database and Backend Connectivity Issues

C.1.1 Issue 1: Connection Refused (WSL/Windows Scenario)

Symptom: The Python CLI, running inside a Windows Subsystem for Linux (WSL) environment, or the React Frontend cannot connect to the Node.js backend running on the Windows host machine. **Cause:** WSL often uses a different internal IP address than the Windows host's external IP address. Connections must target the host machine's physical IP address. **Solution:**

1. Determine the Windows host machine's IP address (e.g., using `ipconfig` in Windows Command Prompt).
2. Update the `API_BASE_URL` variable in the Python CLI's configuration (usually a local `.env` file) to use the host's IP address instead of `localhost` (e.g., change `http://localhost:3000` to `http://192.168.1.5:3000`).

C.1.2 Issue 2: MySQL Access Denied

Symptom: The Node.js backend fails to start, displaying an "Access Denied" error in the console. **Cause:** The credentials (username, password, host) specified in the backend's `.env` file are incorrect or the MySQL user (`pkg_manager`) has not been properly granted permissions. **Solution:**

1. Re-verify the `DB_USER` and `DB_PASS` variables in the backend `.env` file.
2. Confirm the MySQL user permissions were granted correctly by executing the `GRANT ALL PRIVILEGES...` command (see Appendix A).

C.2 Package Data Integrity and Synchronization

C.2.1 Issue 3: Incomplete Dependency Graph

Symptom: The CLI's `install` command fails to resolve a dependency, even though the package exists in the database. **Cause:** The package synchronization process (`POST /api/update-packages`) may have failed or timed out during the initial bulk insertion, resulting in missing entries in the `dependencies` table. **Solution:**

1. As an administrator, access the React Dashboard and manually trigger the **One-Click Package Update** from the Packages View.
2. Monitor the backend logs to ensure the operation completes successfully without any timeout errors before retrying the CLI command.

C.2.2 Issue 4: Package Transition Conflicts (`t64`)

Symptom: The Python CLI detects a `t64` conflict and refuses to install the requested package. **Cause:** The system is attempting to install an incompatible package version

during a package transition (a known issue in certain Debian/Ubuntu releases). **Solution:** The system's Conflict Protection logic is working as intended. The user should:

1. Identify the conflicting package and manually resolve it using an appropriate package management tool (e.g., `apt`).
2. Alternatively, wait for the next data synchronization to occur, which may update the dependency constraints in the database to reflect a safer path.

C.3 User Interface (Frontend) Errors

C.3.1 Issue 5: Downloads View Data Latency

Symptom: New installations performed via the CLI do not immediately appear on the Admin Dashboard's Downloads View. **Cause:** This is usually a frontend caching issue or a slow network connection. **Solution:**

1. Manually refresh the browser page to force a new API call to the `/packagedownloads` endpoint.
2. If running a production build, ensure the Node.js API server has sufficient resources to handle concurrent reads from the large `packagedownloads` table.